

Summary. Machines decide what an image shows, and who owns it, from three sources: the page's alt text, the XMP inside the file, and JSON-LD. This reference shows how to keep all three in agreement across JPG, PNG and WebP, favicons, 3D glTF models and video, and where exporters, encoders and compression break them. It's built on measurements from the game11ty site and the CLO jacket models, with Python and exiftool commands you can run yourself.

Search engines, asset managers and AI models decide what an image shows, and who controls it, from text: the `alt` attribute, the metadata inside the file, and structured data on the page. When those three agree, a machine doesn't have to guess. When they're missing or contradict each other, it guesses, and the guess travels with the image into indexes, datasets and model outputs.

This article explains each layer, the Adobe XMP standard behind embedded metadata, how it's stored in JPG, PNG and WebP, and how rights fields act as a lightweight form of DRM. It ends with a worked example, the game11ty site, and a checklist for agencies and media teams.

Companion article: [Asset Management, Security and AI on the CRM Sync knowledge base](#) — the media manager's view of the same pipeline: what a re-encode destroys and what it protects, raster and mesh compression including Draco and KTX2, and where media should live.

1. Three layers, one description

An image on the web can be described in three places. Each is read by different machines, and only one of them travels with the file.

LAYER	WHERE IT LIVES	WHO READS IT	SURVIVES A DOWNLOAD?
HTML <code>alt</code>	The <code></code> tag on the page	Screen readers, search crawlers, AI models that read the page	No: it stays on the page
Embedded XMP	Inside the image file's bytes	DAMs, Google Images, scrapers, AI training pipelines	Yes, unless a tool strips it
schema.org JSON-LD	A <code><script type="application/ld+json"></code> block on the page	Search engines, knowledge graphs	No: it stays on the page

The page layers describe the image *in context*: what it's doing on this page. XMP describes the image *as an object*: what it shows and who owns it, wherever it ends up. Write the same description into all three. A crawler that reads the page and a pipeline that only ever sees the file then reach the same answer.

2. The Adobe XMP standard

XMP (Extensible Metadata Platform) is Adobe's metadata format, standardised as ISO 16684-1. An XMP block is an RDF/XML document wrapped in an `<x:xmpmeta>` element and stored inside the file as a *packet*. Every Adobe app reads and writes it, and so do exiftool, DAMs and most image libraries.

Each field belongs to a namespace, and the prefix says who defined it. These are the ones that matter for description and rights, with the names the IPTC Photo Metadata Standard 2025.1 gives them:

FIELD (IPTC NAME)	XMP PROPERTY	WHAT IT HOLDS
Alt Text (Accessibility)	Iptc4xmpCore:AltTextAccessibility	Short description, the same as the HTML alt
Extended Description (Accessibility)	Iptc4xmpCore:ExtDescrAccessibility	Longer description for complex images
Description	dc:description	General caption
Creator	dc:creator	Person or organisation that made it
Copyright Notice	dc:rights	The copyright line
Credit Line	photoshop:Credit	How to credit it when published
Web Statement of Rights	xmpRights:WebStatement	URL of the rights or licence page
Licensor URL	plus:Licensor → LicensorURL	Where to license it
Data Mining	plus:DataMining	Whether data mining and AI/ML training are allowed
Digital Source Type	Iptc4xmpExt:DigitalSourceType	How it was made: camera, composite, AI-generated
Edit history	xmpMM:History	Which software saved it, and when

The packet looks like this. It's from the game11ty share image, trimmed:

```

<x:xmpmeta xmlns:x="adobe:ns:meta/">
  <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
    <rdf:Description rdf:about=""
      xmlns:dc="http://purl.org/dc/elements/1.1/"
      xmlns:Iptc4xmpCore="http://iptc.org/std/Iptc4xmpCore/1.0/xmlns/"
      xmlns:xmpRights="http://ns.adobe.com/xap/1.0/rights/">
      <dc:description><rdf:Alt><rdf:li xml:lang="x-default">Inside the OMEN 35L Valorant special e
      <Iptc4xmpCore:AltTextAccessibility><rdf:Alt><rdf:li xml:lang="x-default">Inside the OMEN 35L
      <xmpRights:Marked>True</xmpRights:Marked>
      <xmpRights:WebStatement>https://persephonepunch.github.io/game11ty/rights/</xmpRights:WebSta
    </rdf:Description>
  </rdf:RDF>
</x:xmpmeta>

```

Text fields are `rdf:Alt` lists keyed by `xml:lang`, so one file can carry alt text in several languages. Because the packet is RDF, a parser gets subject–property–value triples, not loose strings.

3. Where XMP lives in JPG, PNG and WebP

The XMP packet is the same in every format. Only its container changes: each format has its own slot for it, next to (not inside) the pixel data.

FORMAT	SLOT FOR XMP	HOW TO RECOGNISE IT	ALSO CARRIES
JPG	<code>APP1</code> segment (marker <code>FF E1</code>) before the image data	Starts with <code>http://ns.adobe.com/xap/1.0/</code> + a null byte	EXIF in another <code>APP1</code> , legacy IPTC in <code>APP13</code> , colour profile in <code>APP2</code>
PNG	<code>iTXt</code> text chunk	Keyword <code>XML:com.adobe.xmp</code>	EXIF in an <code>eXIf</code> chunk
WebP	<code>XMP</code> chunk (the fourth character is a space) in the RIFF container	The <code>VP8X</code> header's XMP flag must be set (<u>spec</u>)	EXIF in an <code>EXIF</code> chunk

I read these slots directly from the game11ty files: every JPG has its XMP in `APP1` and every PNG in `iTXt:XML:com.adobe.xmp`. The share image also has a legacy IPTC `APP13` block, which exiftool keeps in sync with the XMP so older tools see the same copyright.

Because the metadata sits beside the pixels, any tool that re-encodes an image can drop it without changing how the image looks. The usual culprits are image CDNs and optimisers that convert to WebP or AVIF, social platforms, and "export for web" presets set to *None*. A rule of thumb: whenever an image passes through a tool, check that the rights fields came out the other side.

4. How Python reads it: scraping as a semantic breakdown

Scraping a page isn't copying it. A scraper breaks the rendered page into roles: this is a heading, this is an image, this is its description, this block is structured data. The pixels arrive as opaque bytes, and the only thing they can say about themselves is their XMP. So a scraper ends up with two sources of meaning for every image: what the page says about it, and what the file says about itself.

The script below reads all three layers for every image on the live game11ty page, using `requests`, BeautifulSoup and the standard XML parser:

```

import json, re, requests

import xml.etree.ElementTree as ET

from urllib.parse import urljoin

from bs4 import BeautifulSoup

PAGE = "https://persephonepunch.github.io/game11ty/"

NS = {"rdf": "http://www.w3.org/1999/02/22-rdf-syntax-ns#",
      "dc": "http://purl.org/dc/elements/1.1/",
      "Iptc4xmpCore": "http://iptc.org/std/Iptc4xmpCore/1.0/xmlns/",
      "xmpRights": "http://ns.adobe.com/xap/1.0/rights/"}

def xmp(data: bytes) -> dict:
    """Find the XMP packet in JPG/PNG/WebP bytes and read it as RDF."""
    m = re.search(rb"<x:xmpmeta.*?</x:xmpmeta>", data, re.S)
    if not m:
        return {}

    descs = ET.fromstring(m.group()).findall(".//rdf:Description", NS)

    def text(prop):
        # a property can sit in any Description,
        pre, name = prop.split(":")
        # as an element or as an attribute
        for d in descs:
            el = d.find(prop, NS)
            if el is not None:
                li = el.find(".//rdf:li", NS)
                return (li if li is not None else el).text
            if d.get(f"{{{NS[pre]}}}{name}"):
                return d.get(f"{{{NS[pre]}}}{name}")
    return {"alt": text("Iptc4xmpCore:AltTextAccessibility"),
            "rights": text("dc:rights"),
            "webStatement": text("xmpRights:WebStatement")}

```

```

soup = BeautifulSoup(requests.get(PAGE).text, "html.parser")

ld = {o["contentUrl"]: o for s in soup.find_all("script", type="application/ld+json")
      for o in json.loads(s.string).get("@graph", [])}

for img in soup.find_all("img"):
    url = urljoin(PAGE, img["src"])

    page_alt = img.get("alt")                # layer 1: the page
    file_meta = xmp(requests.get(url).content) # layer 2: the pixels' own XMP
    graph = ld.get(url, {})                  # layer 3: JSON-LD

```

Run against the live site on 3 October 2026, it found all three layers on every one of the 8 raster images, with matching descriptions and a rights URL in each file.

Two things in this code generalise to any metadata parser:

- **Treat XMP as RDF, not text.** The same property can be written as an element or an attribute, and exiftool spreads properties across several `rdf:Description` blocks, one per namespace. My first version read only the first block and reported every rights field as missing.
- **The page and the file are read separately.** `img.get("alt")` comes from the DOM; `xmp()` comes from bytes fetched on their own. A dataset built from downloaded images only ever sees the second.

Crawling: BFS, DFS and the files that set the rules

Before a scraper can break a page into roles, it has to find the pages.

game11ty's `sync.py` does this with a **BFS crawl**, short for *breadth-first search*: it visits a site level by level, starting from the home page.

1. Put `/` in a queue.
2. Take the page at the front of the queue and download it.

3. Collect its internal links, and add any page not seen before to the back of the queue.
4. Repeat until the queue is empty or the page limit is reached (200 in `config.py`).

```
Level 0:  /  
Level 1:  /about    /products    /contact          ← linked from the home page  
Level 2:  /products/omen-35l    /products/headset ← linked from level 1
```

The alternative is **DFS**, *depth-first search*: follow one link, then the first link on that page, as deep as possible before backing up. On a site with a blog or long pagination, DFS can spend its whole budget in one corner. BFS reaches the main pages first, finds each page by its shortest route, and can't be trapped by one long chain of links. The "seen" list stops pages that link to each other from being fetched twice.

A crawl copies what's **reachable by links**, not the site's file tree; a web server never shows its files. Pages nothing links to are missed unless they're listed in `sitemap.xml` or seeded in `config.py`, uploads no page uses are never copied, and CMS data, drafts and password-protected pages stay behind.

The files that tell crawlers the rules. Four conventions let a site describe how crawlers and AI should treat it:

FILE OR TAG	WHERE IT LIVES	WHAT IT SAYS	DOES IT ENFORCE ANYTHING?
<code>robots.txt</code>	<code>/robots.txt</code> at the domain root (RFC 9309)	Which paths each crawler may fetch, by name, including AI crawlers	No. Well-behaved crawlers obey it; hostile ones ignore it, and listing a private path advertises it
<code>nofollow</code>	<code>rel="nofollow"</code> on a link, or <code><meta name="robots" content="nofollow"></code> for a whole page	Don't follow this link. Google also reads <code>sponsored</code> and <code>ugc</code>	No. Google treats it as a hint, and the page can still be found through sitemaps or other links (Google)
<code>security.txt</code>	<code>/.well-known/security.txt</code> (RFC 9116 , 2022)	Where to report a vulnerability. <code>Contact</code> and <code>Expires</code> are required	No. It's a contact card for security researchers
<code>llms.txt</code>	<code>/llms.txt</code> , or under a subpath	A Markdown summary of the site with links to versions written for AI, such as Markdown pages. A 2024 proposal by Jeremy Howard, not a standard (llmstxt.org)	No. It's an invitation, not a permission

The security point: all four are declarations, not locks. They steer the crawlers that choose to listen, so use them for that: `robots.txt` and `nofollow` to keep good crawlers on the right pages, `security.txt` so people can report problems, `llms.txt` to point AI at clean, rights-labelled content. Never put secrets or private paths in them, since they're public. What actually keeps a crawler out is the allow list, sign-in and encryption described in later sections.

game11ty deliberately has none of these files: it's a demo, not meant to be found or ranked. They're here because a developer making content for AI security needs the concepts. Know which signals crawlers read, that `robots.txt` and `security.txt` must sit at the domain root while `llms.txt` can live under a subpath, and that none of them replaces real access control. game11ty's own `sync.py` doesn't read `robots.txt`, which is fine for crawling your own site; a crawler of other people's sites must respect it.

5. Optimization and bit size

Across the 15 images on game11ty, metadata adds 56 KB to 4.1 MB, or 1.4%. That cost is fixed per file, not per pixel, so it barely registers on large photos and dominates small icons. These are the measured figures:

FILE	SIZE	METADATA	SHARE
HyperX hero PNG, 2880 px wide	2,031 KB	3.4 KB	0.2%
OMEN desktop PNG	633 KB	3.5 KB	0.6%
Gear photo JPG	160 KB	5.2 KB	3.2%
Share image JPG	99 KB	7.1 KB	7.1%
32 px favicon PNG	5.3 KB	2.5 KB	47%
Red emblem PNG	6.1 KB	3.8 KB	63%

Three things shape that cost:

- **Padding.** Adobe tools and exiftool leave empty space inside a JPEG's XMP packet so it can be edited later without rewriting the file. In the share image that padding is 2,498 bytes, about 40% of its packet. The PNG packets carry only 74 bytes.
- **Edit history.** `xmpMM:History` and document IDs record every save. They are useful in a DAM and dead weight on the web.
- **What you actually need.** Stripping the 32 px favicon to bare pixels gives 2,821 bytes; adding back only alt text, copyright and the rights URL gives 3,818. A minimal rights set costs about 1 KB.

Frame load. In both JPG and PNG the metadata sits *before* the pixel data: the `APP` segments come before the JPEG scan, and the `iTXt` chunk came before `IDAT` in every PNG checked. A browser decodes nothing until those bytes have arrived, so on a slow connection every kilobyte of metadata delays the first

painted row. All four JPGs here are *baseline*, which decode top to bottom as bytes arrive; a *progressive* JPEG paints a coarse full frame first. Either way, a lean packet means the first pixels arrive sooner.

Bit depth. Every PNG here is 8 bits per channel. A 16-bit PNG doubles the pixel data and adds nothing a screen can show, so check for it before worrying about metadata.

Conversion strips by default. Converting the 101 KB gear photo to WebP with Google's `cwebp` gave 28 KB and **no metadata at all**: the alt text and rights were gone. With `-metadata xmp` the result was 33 KB and both survived. Other encoders and optimisers, including Rust-based tools such as `oxipng`, also drop metadata unless told to keep it, so set the keep option explicitly in your build.

Rust and the pixel buffer. When Rust handles an image, it works on a *pixel buffer*: an array of integer channel values, usually one unsigned byte (`u8`, 0–255) per red, green, blue and alpha channel. Each stage has its own name:

STAGE	TERM	IN RUST
Reading a JPG, PNG or WebP into pixel values	Image decoding; <i>memory-safe decoding</i> when done in Rust	The compiler's ownership and bounds checks rule out the buffer overflows that C decoders are prone to
Holding the values in memory	Pixel buffer or framebuffer	<code>ImageBuffer</code> of <code>Rgba<u8></code> in the <code>image</code> crate
Storing each channel as a whole number	Integer pixel format	<code>u8</code> per channel; <code>u16</code> or <code>f32</code> for higher bit depth
Turning shapes or 3D geometry into pixels	Rasterization; <i>software rendering</i> on the CPU	Fills the buffer directly
Painting a <code><canvas></code> on each frame from WebAssembly	Rendering into WASM linear memory	Rust writes the bytes, JavaScript passes them to the canvas as <code>ImageData</code> ; <i>zero-copy</i> when no copy is made

None of these stages handles XMP. The metadata sits beside the pixel data, so a pipeline that decodes to a pixel buffer and encodes again keeps only the

pixels, as the `cwebp` test showed. A Rust image pipeline has to read the XMP before decoding and write it back after encoding, or the rights are lost at the first conversion.

6. Security note

Pixels and tokens. Images and text are split into discrete integer units before a model uses them. A pixel stores brightness (0–255 per channel); a token stores a vocabulary index. Inside a model both become embedding vectors, which is how multimodal models compare images and language. Rust is common in the pipeline, including Hugging Face's tokenizers, because its compiler enforces memory safety.

Parsers are the attack surface. Many decoders for SVG and 3D models (glTF, Draco, OBJ, FBX) are written in C or C++, and a malformed file can exploit a memory bug in them. SVG can also carry scripts. Protect these files on three layers:

- **Code:** parse untrusted files with memory-safe code such as Rust, or sandbox C/C++ decoders.
- **Content:** sign assets (e.g. C2PA) to prove owner and integrity. XMP rights fields can be edited, so they aren't proof.
- **Transit:** serve over TLS. It protects files in transit only, so it complements signing.

AI agents should check signatures and rights before trusting a file.

Two 2026 incidents, one lesson. In the OpenAI–Hugging Face breach, OpenAI's own test agents got in through a flaw in an HDF5 dataset parser ([Wikipedia](#)). Separately, in July 2026 researchers at Hacktron reached OpenAI's community forum through a heap overflow in libheif, an image parser: the forum software's

image check couldn't identify HEIF files, so it handed them to ImageMagick, "exposing the underlying libheif parser directly to attacker-controlled files" ([Hacktron](#)). Different parsers, same lesson. ImageMagick is very likely the software resizing your images; the companion article's [ImageMagick](#) section lists what a media manager should require of it.

Higher-risk assets. DAM libraries, 3D models and firmware need extra care: verify by hash or signature, restrict publishing, scan before processing, and install firmware only with a valid signature (secure boot).

7. Release scan

game11ty now gates every deploy on an asset scan. `asset_scan.py` runs in a throwaway Docker container with no network, a read-only file system and no secrets, so a hostile file has nothing to reach. The site only builds if the scan passes.

ASSET	WHAT IT CHECKS
Every file	Real type matches the extension, read from the file's first bytes (its "magic bytes", e.g. <code>89 50 4E 47</code> = PNG); no executables, even disguised
JPG / PNG / WebP	Valid file structure; rights XMP present (<code>WebStatement</code> , alt text)
SVG	Valid XML; no <code><script></code> , <code>on...=</code> handlers, <code>javascript:</code> links, <code><foreignObject></code> , external URLs or XML entities
glTF / GLB	Header and chunks valid; every <code>bufferView</code> and <code>accessor</code> inside its data; buffer paths can't escape the model folder; <code>asset.copyright</code> set and not an exporter's default
PDF	No JavaScript, auto-run, launch or embedded-file actions

On a test set of deliberately bad files it caught all 13 planted problems: a PNG renamed `.jpg` , an executable named `.png` , an image with no rights data, a

hostile SVG, a PDF with auto-running JavaScript, and a GLB doctored so its data pointers run past the end of the file. The real CLO avatar fails too, on Stager's "2025 (c) Adobe Inc." copyright, which is the point: it shouldn't ship with that claim.

After the build, CI publishes `asset-manifest.json`, a SHA-256 hash of every released file, so a download can be checked against what was scanned.

The lesson behind it is the security note's. In the July 2026 OpenAI–Hugging Face incident, the way in was a flaw in an HDF5 dataset parser running with access to credentials ([Wikipedia](#)). The scan parses untrusted files where nothing can be reached, and treats a file whose content doesn't match its label as an error. Run it locally with `python3 scripts/asset_scan.py src`.

The ImageMagick case shows why the type check is an allow list. The forum's check couldn't recognise HEIF, so instead of rejecting the unknown file it passed it on to a heavier parser. The scan does the opposite: a file type that isn't on its list is an error, never skipped. Writing that rule as a test first showed the scan had been silently skipping unlisted types; it now blocks them, and a mutation test proves it. Hacktron's advice matches the rest of the design: disable formats you don't need, such as HEIF and AVIF, and run image processing in hardened, short-lived sandboxes, using ImageMagick's security policy to restrict accepted formats ([Hacktron](#)).

Magic bytes: auditing at the core

Most file formats start with a fixed signature, its *magic bytes*. Software reads them to learn what a file really is, whatever its name says. The extension is a label anyone can change; the magic bytes are part of the content.

FORMAT	FIRST BYTES (HEX)	AS TEXT
PNG	89 50 4E 47 0D 0A 1A 0A	%PNG
JPEG	FF D8 FF	
GIF	47 49 46 38	GIF8
WebP	52 49 46 46 ... 57 45 42 50	RIFF ...WEBP
PDF	25 50 44 46 2D	%PDF-
glTF binary (GLB)	67 6C 54 46	glTF
ZIP (also .docx, .pptx)	50 4B	PK
Windows program	4D 5A	MZ

Each of these is a run of `u8` integers, 0–255: the same type Rust uses for pixel channels. In Rust, detecting a type this way is called *format detection* or *file type sniffing* (`image::guess_format`, the `infer` crate). The bytes are the same in any language, so the game11ty scan does it in Python.

Why auditing at the core is the secure default. Every layer above the bytes is a claim: the file name, the extension, the server's `Content-Type`, an XMP field, a page's `alt` text. Any of them can be wrong or forged. The bytes are what a decoder actually executes on, so an audit that starts there can't be fooled by a relabelled file. It catches:

- **Disguised executables:** a program named `photo.png` still starts with `MZ`.
- **Mislabelled files:** game11ty's Webflow favicons were PNG data named `.jpg`. GitHub Pages sets `Content-Type` from the extension and sends no `X-Content-Type-Options: nosniff` header, so those files were served as `image/jpeg`, and each browser guessed the real type itself.
- **Wrong-parser attacks:** a file routed by its label to the wrong decoder lands in code that wasn't built for it, which is where memory bugs get triggered.

Magic bytes only prove what kind of file it is, not that it's safe. A file can carry a valid signature and still be malformed or hostile, which is why the scan goes on to check structure, bounds and active content. The order matters: confirm the type from the bytes first, then validate it as that type.

See it for yourself. On a Mac or Linux terminal, `xxd -l 16 file.png` prints a file's first 16 bytes, and `file file.png` names the type it finds:

```
$ xxd -l 16 favicon256.png
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452  .PNG.....IHDR
$ file favicon256.png
favicon256.png: PNG image data, 32 x 32, 8-bit/color RGBA, non-interlaced
```

Two details trip people up. The signature isn't always at byte 0: WebP starts with `RIFF` and only says `WEBP` at byte 8, which is why signature tables list an *offset*. And a match proves the type, not that the file is safe.

Further reading, simplest first:

- [Wikipedia: List of file signatures](#): the most cited table, with hex, text and offset for hundreds of formats
- [Gary Kessler's File Signatures Table](#): the long-standing digital-forensics reference, listed by extension
- [MDN: MIME types](#): plain-language guide to `Content-Type`, and why it can disagree with the bytes
- [WHATWG MIME Sniffing standard](#): the official rules browsers follow when they guess a type from the bytes
- `man file` and `man magic`: built into macOS and Linux; the `file` command's signature database is called *magic*, which is where the name comes from

8. Publish and unfurl protection: a spec for the CRM Sync stack

This section is a plan, written as an agile spec that people and AI assistants can both work from. It applies the release scan to assets published through CRM Sync, which runs on Cloudflare and Xano. game11ty's scan is the working reference; the rest is not built yet.

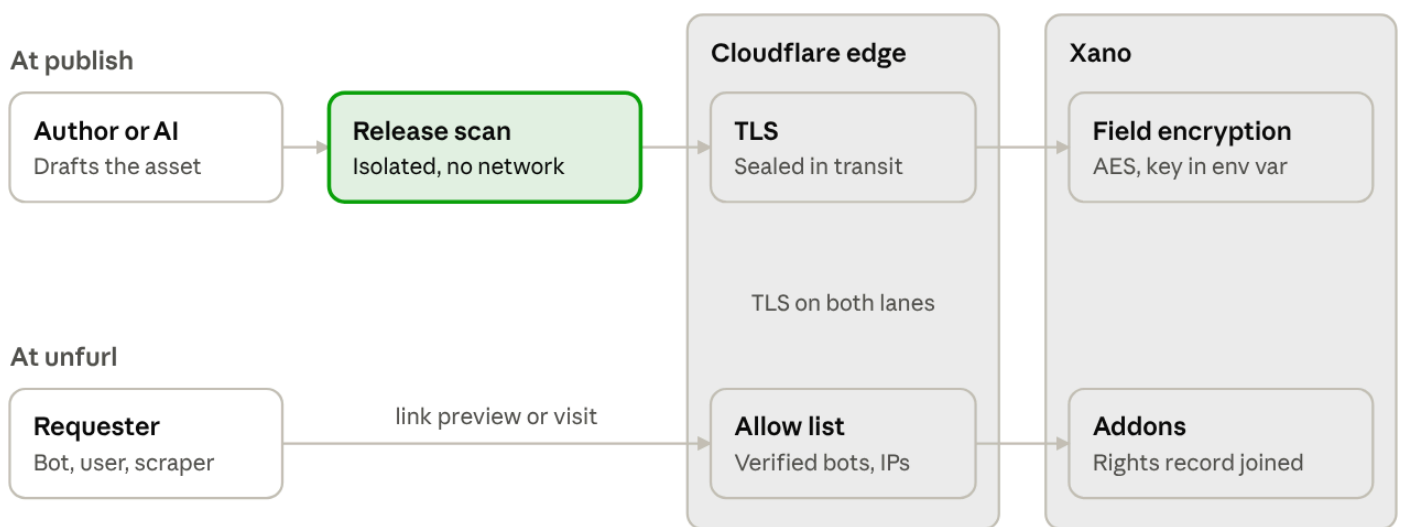
In plain words

A file is at risk at two moments. **At publish**, when it goes live: a bad or mislabelled file could slip out. **At unfurl**, when someone pastes a link into Slack, iMessage, LinkedIn or X and that app fetches the page and its preview image: anyone can pretend to be one of those apps to scrape files. The protection also has to **hold as the system grows** ("scales horizontally"): more servers, regions and customers, with no gaps between them.

LAYER	WHAT IT DOES, IN PLAIN WORDS	TOOL
Allow list	A guest list at the door: only known visitors get in	Cloudflare WAF custom rule with an IP or bot list (Cloudflare)
TLS	A sealed envelope for data while it travels	Cloudflare, on every connection
Field encryption	A locked drawer for sensitive columns in the database	Xano Encrypt filter, AES, key kept in an environment variable (Xano)
Addons	Staples each asset's rights record to every answer the API gives	Xano Addons, reusable related-data queries (Xano)
Release scan	Inspects every file before it goes live	<code>asset_scan.py</code> , in an isolated container

TLS protects data only while it moves, and encryption at rest protects it only while it's stored, so CRM Sync needs both. Its docs already store credentials in Cloudflare's encrypted key-value store, one key per customer, masked in every API response.

Five layers guard each file at publish and at unfurl



Green: built today in game11ty. The other layers are planned for CRM Sync.

Scale-out: the same rules run at every Cloudflare location and for every Xano tenant.

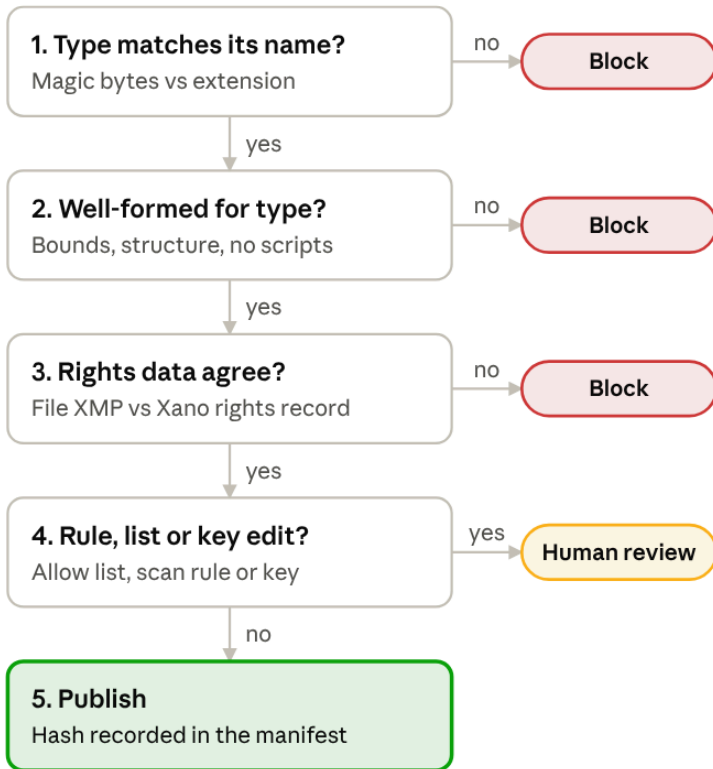
A file enters on the publish lane and is scanned before it reaches Cloudflare and Xano; a request enters on the unfurl lane and meets the allow list first.

Decision ladders

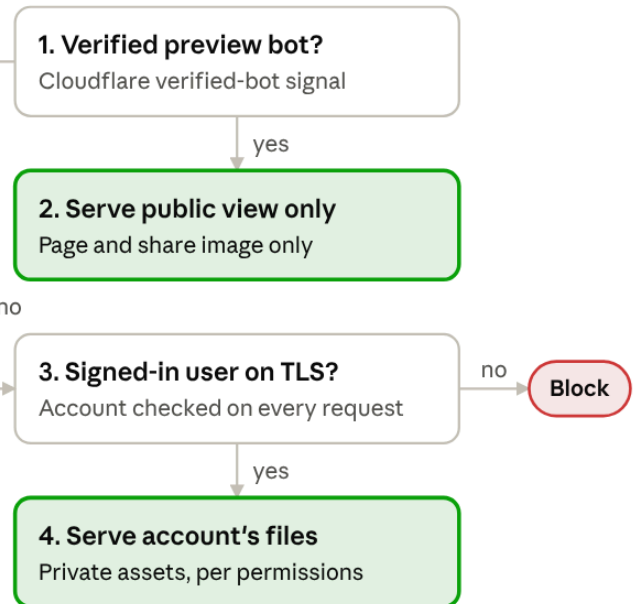
A decision ladder is a fixed set of questions asked in order. A file or request climbs one rung at a time and stops at the first "no", so every outcome has a known reason.

Every file and request stops at the first failed rung

At publish



At unfurl



Green boxes are the only ways through; every other path ends at a block or a human review.

At publish:

1. Does the file's real type (magic bytes) match its name? No → block.
2. Is the file well-formed for that type: bounds, structure, no active content? No → block.
3. Does it carry rights data, and do the Xano rights record and the file agree? No → block.
4. Did this release change a rule, an allow list or a key? Yes → hold for human code review.
5. Publish, and record the file's hash in the manifest.

At unfurl:

1. Is the request on the allow list as a verified preview bot (Slack, Apple, LinkedIn, X), checked with Cloudflare's verified-bot signal, not just the name it claims? No → go to rung 3.
2. Serve the public page and its share image only. Nothing private, no originals.
3. Is it a signed-in user over TLS? Yes → serve what their account allows. No → challenge or block.

The ladders run the same way at every Cloudflare location, so adding servers or customers adds no new rules to maintain.

User stories and tests (TDD)

Each story is written with its test first, in test-driven development (TDD) style. The code is done when its tests pass.

STORY	GIVEN	WHEN	THEN
As a publisher, I want mislabelled files stopped	A PNG named <code>.jpg</code>	It is published	The release is blocked and the report names the file
As a rights owner, I want my claim on every copy	An asset with a Xano rights record	Any API returns it	The rights record is attached through an Addon
As a brand, I want link previews to work	A verified Slack preview bot	It fetches the article	It gets the page and share image, nothing else
As a security lead, I want scrapers kept out	A bot that only claims to be Slackbot	It fetches an original asset	It is challenged or blocked
As an operator, I want sensitive fields unreadable	A licence contact field in Xano	Someone reads the raw table	They see ciphertext, not the value

Code review gates

AI can draft any change. A person approves the ones that change who gets in or how data is locked:

CHANGE	AI MAY	HUMAN MUST
New asset or article	Draft and run the scan	Nothing extra if the scan passes
Scan rule	Draft the rule and its test	Review and approve
Allow list entry	Propose it with evidence	Approve; entries expire and are re-reviewed
Encryption key or algorithm	Never	Rotate keys and approve
Rights record	Draft from file metadata	Confirm the owner

Keep the spec, its tests and the review record in the repository next to the code, so every change can be traced to a story, a test and an approver.

Magic bytes across CRM Sync: Shopify, WordPress, Webflow and Xano

In CRM Sync, the magic-byte check belongs in one place: the Cloudflare Worker that every upload and every file request already passes through. Not in a Shopify theme, a WordPress theme, a Webflow template or a page script. That makes the protection **theme-agnostic**: a merchant can switch Shopify or WordPress themes or redesign a Webflow site, and every file is still checked, because the check never lived in the theme.

In plain words: the theme decides how a file *looks* on the page. The Worker decides whether the file is *allowed to exist* and how it is *labelled* when it's served. Keeping those apart means a design change can't switch security off.

Upload path, the same for every platform:

1. A file arrives at the Worker: from the Shopify app, a WordPress plugin, a Webflow Designer extension, or a Xano admin screen.
2. The Worker reads the first bytes and detects the real type. The type must be on that destination's allow list, e.g. images and GLB for products, PDF for documents. Executables are never allowed.

3. The Worker records the detected type, the SHA-256 hash and the rights fields in the asset's Xano row, with sensitive fields encrypted.
4. Only then does the Worker request an upload slot from the platform. Shopify and Webflow upload in two steps and WordPress in one; either way the check runs before anything reaches them, and the file name's extension is set from the *detected* type, never the original name.
5. When the file is served through Cloudflare, the Worker sets `Content-Type` from the recorded type and adds `X-Content-Type-Options: nosniff`, so browsers trust the checked label instead of guessing. That's the header GitHub Pages doesn't send for game11ty.

Forms have MIME types too. An HTML form declares how it sends data in its `enctype`, which is itself a MIME type:

FORM ENCODING (MIME TYPE)	USED FOR	WHAT TO TRUST
<code>application/x-www-form-urlencoded</code>	Plain text fields; the default	Validate each field's value
<code>multipart/form-data</code>	Any form with a file upload	Each file part carries its own <code>Content-Type</code> , guessed by the browser from the file name. It's a label from the uploader, so the Worker ignores it and reads the magic bytes
<code>text/plain</code>	Rare; debugging	Treat as untrusted text
<code>application/json</code>	<code>fetch()</code> calls from apps and extensions, not HTML forms	Validate against a schema

So the Worker checks the request's form type first, then each file part's real bytes. A part labelled `image/png` that starts with `MZ` is rejected, whatever the form said.

PLATFORM	WHERE FILES ENTER	UPLOAD STEPS	WHAT THE WORKER CONTROLS
Shopify	App, Admin API	Two: <code>stagedUploadsCreate</code> , then <code>fileCreate</code>	The bytes and type before a staged target is requested
WordPress	Plugin, REST API	One: <code>POST /wp-json/wp/v2/media</code> , file in the body, name in <code>Content-Disposition</code>	The file name and <code>Content-Type</code> sent with the upload
Webflow	Designer extension, Data API	Two: <code>create asset</code> (file name and MD5), then upload to a presigned URL	The file name's extension and the hash Webflow checks
Xano	Admin screens, API	Stores the asset row	Detected type, hash, rights record (Addon), encrypted fields
Cloudflare	Every request	Not applicable	<code>Content-Type</code> , <code>nosniff</code> , allow list, TLS

Plain POST and GET to Xano. Between the Worker and Xano there's no SDK or extra security vendor, just HTTPS requests: POST to send data such as uploads and rights records, GET to read it. Xano is still a hosted service, so the security comes from what each endpoint enforces: sign-in, input checks, Addons and encrypted fields. Two rules keep it safe. Never put secrets or personal data in a GET URL, because servers and browsers log URLs; send them in a POST body. And keep the Xano API key in the Worker's secrets, never in a theme or page script, so it stays protected whatever theme or site builder is in front.

Design to theme: one backend for every template engine

The Worker and Xano sit below the template layer. Whether a page is built with Liquid, PHP, Astro or React, the template only produces HTML and URLs; the checks, allow list, encryption and rights records all run in the Worker and Xano over HTTPS. Swap the front end and the backend rules stay the same. What changes is *when* each front end talks to the Worker, and where its secrets must live:

FRONT END	TEMPLATE ENGINE	WHEN PAGES RENDER	HOW IT REACHES THE WORKER	KEEP SECRETS IN
Shopify	Liquid	Every request, on Shopify's servers	Liquid can't make HTTP calls, so through an <u>app proxy</u> : Shopify forwards a store URL to the Worker with a signature the Worker verifies, and a reply sent as <code>application/liquid</code> renders inside the theme. Or browser calls	Worker secrets only; nothing in the theme
WordPress	PHP templates and blocks	Every request, on the WordPress server	Server-side with the <u>HTTP API</u> (<code>wp_remote_get</code> , <code>wp_remote_post</code>), or browser calls	<code>wp-config.php</code> or server environment, never theme files
Jekyll	Liquid	Once, at build time	The build fetches and bakes data in; browser calls for live data	Build environment; never in the <code>_site/</code> output
Eleventy	Liquid, Nunjucks and others	Once, at build time	Data files fetch at build; browser calls for live data	Build environment; never in the output
Astro	Astro components	At build by default; per request with server rendering	Fetch in the component script at build or request time; browser calls	Unprefixed variables; anything named <code>PUBLIC_</code> reaches the browser (<u>Astro</u>)
Next.js	React	At build, or per request with server components	Server-side fetch; browser calls	Unprefixed variables; anything named <code>NEXT_PUBLIC_</code> is written into the browser bundle (<u>Next.js</u>)

The rule is the same in every row: the Worker's URL can appear in a template, but its keys never do. One caution for the build-time front ends (Jekyll, Eleventy,

static Astro and Next.js): data fetched at build is frozen into the HTML until the next build. Anything that must stay current or private, such as a rights change or a signed-in user's files, should be fetched live through the Worker, not baked in.

Head code: in the theme, or a higher-order function in the Worker

Head code is everything placed in a page's `<head>`: scripts, tracking tags, a content security policy, meta tags. The usual place for it is the theme: Shopify's `theme.liquid`, WordPress's `header.php`, Webflow's custom-code box, an Eleventy or Jekyll layout. The security best practice is to keep only the **look** there and move security into a **higher-order head function** in the Cloudflare Worker.

In plain words: a higher-order function is a function that wraps another one. The Worker's head function takes whatever page the origin sends, from any platform, and returns it hardened: security headers set and the security baseline script added. It's the same idea as Helmet, the Express middleware that sets headers such as `Content-Security-Policy` and `Strict-Transport-Security`, but at the edge and for every platform at once.

	HEAD CODE IN THE THEME	HIGHER-ORDER HEAD FUNCTION IN THE WORKER
Where it lives	Inside each theme or template	One function at the Cloudflare edge
Who can change it	Anyone with theme or design access	Only a reviewed Worker deploy (the code review gates)
Theme switch or redesign	Lost, or copied by hand into the new theme	Unaffected
Security headers	Only as <code><meta></code> tags, and some (<code>frame-ancestors</code> , HSTS) don't work that way	Real HTTP headers on every response
Same on Shopify, WordPress, Webflow, Astro, Next	No: a separate copy per platform	Yes: one source
Secrets	Easy to paste in by mistake, and public once there	Stay in the Worker's secrets

In the Higher-Order Stack, the layering CRM Sync is built on, this is a clean split of layers. The theme head carries Layer 2, the per-brand **look**: `theme.css`, fonts, brand tokens. The Worker carries Layer 1, Tier A, the always-on regulatory **baseline**: security headers, the consent gate and the integrity-checked loader that brings in Tier B scripts only where they're needed. It runs on every page and fails closed.

A sketch of the head function. It's an illustration of the pattern, not the CRM Sync code:

```
// Higher-order: wraps any fetch handler and returns a hardened response.
const withHead = (handler) => async (request, env, ctx) => {
  const res = await handler(request, env, ctx)
  const out = new Response(res.body, res)
  const nonce = crypto.randomUUID()
  out.headers.set("Content-Security-Policy", `default-src 'self'; script-src 'self' 'nonce-${nc
  out.headers.set("Strict-Transport-Security", "max-age=31536000; includeSubDomains")
  out.headers.set("X-Content-Type-Options", "nosniff")
  out.headers.set("Referrer-Policy", "strict-origin-when-cross-origin")
  if (!out.headers.get("content-type")?.includes("text/html")) return out
  return new HTMLRewriter()
    .on("head", { element(el) {
      el.append(`<script src="/embed/stack-loader.js" nonce="${nonce}"></script>`, { html: true
    } })
    .transform(out)
}

export default { fetch: withHead((request) => fetch(request)) }
```

Cloudflare's `HTMLRewriter` edits the HTML as it streams, so the theme never needs to know the baseline exists.

Where the Worker can't sit in front. Shopify doesn't support a Cloudflare proxy in front of a storefront domain; records pointing to Shopify must be DNS-only ([Cloudflare community](#)). There, the Worker hardens everything it serves itself (the app proxy route, embeds, APIs and files), and the theme head shrinks to a single loader tag. Check each host's proxy policy before relying on the Worker for its pages.

This part of the spec isn't built or checked against the CRM Sync code yet; it describes where the check should sit.

9. Connected devices: the machine plane

A connected product, such as a household thermometer or a printer's ink monitor, adds a fourth plane to the client, server and commerce planes: **the machine**, with its own network identity. Commerce, theme and server security don't protect it, and it can't protect them. Like section 8, this is a plan for CRM Sync; the parser below is the working reference.

PLANE	WHERE IT RUNS	WHO CONTROLS IT	WHAT PROTECTS IT
Client (theme)	The shopper's browser	Anyone; it's fully public	Nothing it does is trusted. Look only, no secrets
Commerce	Shopify	Shopify	Shopify's own security; Functions in their WebAssembly sandbox
Server	Cloudflare Worker + Xano	You	Allow list, TLS, field encryption, release scan, edge head function
Machine	The device, in someone's home, on their network	You ship the code; the owner holds the hardware	Memory-safe firmware, secure boot, signed updates, a unique identity per device

In plain words: after a device ships, you can't see or reach the hardware. Anyone can open it, read its memory or send messages pretending to be it. So the server treats every reading as untrusted input, exactly like an uploaded file.

What Shopify covers, and what's additional

FEATURE	BUILT INTO SHOPIFY?	WHAT YOU ADD
Shopify Functions (discounts, shipping, payment, checkout validation)	Yes: WebAssembly, with Rust as a first-class language (Shopify)	Nothing. But a Function only sees the cart data Shopify sends it (at most 128 kB); it can't receive a file or run on a device
3D product media	Partly: GLB and USDZ only, converted and optimized by Shopify above 15 MB (Shopify Help)	A scan before upload, and the rights record kept in Xano, because a re-encode can drop embedded metadata
Mechanical 3D / CAD (STEP, IGES, STL, 3MF)	No	Your own pipeline: Worker, a sandboxed parser, Xano; a converted GLB for display
Machine firmware	No; Shopify can sell it, never run or check it	Device code, signing, secure boot, updates, entitlement-gated delivery

What a connected device needs

1. **On the device:** firmware in Rust `no_std` (no operating system, no heap), with the chip's memory protection unit; secure boot so it only runs signed firmware; signed over-the-air updates.
2. **A unique identity per device:** never a shared key or default password. Cloudflare's mutual TLS checks a client certificate per device, built for hardware that can't sign in like a person ([Cloudflare](#)). One stolen device can be revoked without touching the rest.
3. **At the Worker:** check every reading against a schema, reject values outside physical range, rate-limit and allow-list devices. A reading is a file.
4. **In Xano:** readings stored with sensitive fields encrypted; owner, consent and entitlement joined through Addons.
5. **In commerce:** the device never holds Shopify keys. It reports; the server decides.

Example: an ink monitor that reorders.

```

Printer —mTLS→ Worker → Xano → rule → Shopify
"ink 8%"      checks cert,   stores    ink < 10%,    order created
(signed)      schema, range  reading   owner consented, by the server
                                   and owner  under spend cap

```

If the printer is compromised, the worst it can do is send a false low-ink reading; the order still needs the owner's consent and stays under the spending cap they set. The device triggers, the server authorizes, the person sets the limits. A **household thermometer** adds privacy: room temperatures over time can show when a home is occupied, so treat them as personal data, collect only what the feature needs, and encrypt them at rest.

Memory safety on real hardware

On a microcontroller, Rust's safety comes mostly from the compiler, at no run-time cost: ownership and borrowing stop use-after-free and data races; bounds-checked reads (`slice.get()`) return "nothing" instead of memory past the end; checked arithmetic stops size calculations from wrapping on 32-bit chips. Hardware registers are modelled as values that can be taken only once, `unsafe` is confined to a small hardware-access layer, and a panic halts or resets the device instead of running corrupted. Underneath, the chip's memory protection unit and a stack-overflow guard add hardware enforcement.

The reference implementation is a `no_std` parser for the GLB format, in `firmware/glb-parse`. It has no `unsafe` code and borrows its result from the input, with no copy or allocation:

```

#![cfg_attr(not(test), no_std)]    // no operating system, no heap

pub fn glb_json(data: &[u8]) -> Result<&[u8], GlbError> {
    if data.get(0..4) != Some(b"glTF") { return Err(GlbError::NotGlb); }
    if u32_at(data, 8)? as usize != data.len() { return Err(GlbError::LengthLie); }
    let len = u32_at(data, 12)? as usize;
    if data.get(16..20) != Some(b"JSON") { return Err(GlbError::NotGlb); }
    let end = 20usize.checked_add(len).ok_or(GlbError::LengthLie)?; // can't wrap around
    data.get(20..end).ok_or(GlbError::LengthLie)                  // can't read past the end
}

```

TEST	WHAT A TYPICAL C PARSER RISKS	RESULT
Valid file	—	JSON returned
Program bytes (<code>MZ</code>) instead of a model	Parsed as a model	<code>NotGlb</code>
Empty input	Reads uninitialised memory	<code>NotGlb</code>
Header cut off	Reads past the end	<code>Truncated</code>
Header lies about total size	Trusts the lie	<code>LengthLie</code>
First chunk isn't JSON	Parses binary as JSON	<code>NotGlb</code>
Chunk claims one byte more than exists	Buffer overflow	<code>LengthLie</code>
Chunk length <code>0xFFFFFFFF</code>	<code>20 + len</code> wraps to 19 on a 32-bit chip and passes the check	<code>LengthLie</code>

CI runs the 8 tests and a mutation check (5 of 5 mutants killed), then builds the same code for two targets: `thumbv7em-none-eabihf`, an Arm Cortex-M4/M7 microcontroller, and `wasm32-unknown-unknown`, for a Worker checking uploads. One

codebase guards the device and the edge. Tools cover what the compiler can't: Miri finds undefined behaviour in `unsafe` code, Kani proves properties for every input, `cargo-fuzz` throws millions of mutated files at a parser, and `cargo-geiger` counts `unsafe` code in dependencies.

The limits: registers and C libraries still need `unsafe`, which Rust shrinks but doesn't remove; logic bugs and timing side channels aren't memory bugs; a C decoder linked into Rust keeps C's risks.

The law now covers these devices

- **EU Cyber Resilience Act:** since 11 September 2026, manufacturers must report actively exploited vulnerabilities and severe incidents, with an early warning within 24 hours and a full notification within 72. The full security requirements, CE marking and conformity assessment apply from 11 December 2027. Consumer IoT is in scope ([European Commission](#)).
- **UK product security regime (PSTI):** in force since 29 April 2024. Universal default and easily guessed passwords are banned, and smart thermostats and appliances are named as covered products; manufacturers, importers and retailers share the duties ([GOV.UK](#)).

Selling a connected thermometer or printer into the EU or UK makes device security a legal obligation, whether it's sold through Shopify or not.

10. Concurrency, permissions and regulated data: an AI-shaped design

This section is also a plan. It settles four questions every layer above has to answer the same way: what Rust can and can't prevent, where permissions live, where each class of regulated data may go, and how infrastructure changes when AI agents act inside it.

Data races and race conditions

Rust prevents one kind of timing bug and not the other.

	DATA RACE	RACE CONDITION
What it is	Two threads touch the same memory at once, at least one writing, with no coordination; memory is corrupted	The outcome depends on timing, even though memory stays safe
Example	A device's sensor task and network task write one buffer at the same time	Two "ink 8%" readings a second apart both trigger a reorder
Prevented by Rust?	Yes, at compile time. Ownership and the <code>Send</code> / <code>Sync</code> rules reject unsafe sharing	No. It's logic across requests, often across machines
Who fixes it	The compiler	The database: an idempotency key with a unique index (one reorder per device per low-ink event) and conditional updates ("only if no open reorder exists")

HOW OWNERSHIP EXCLUSION WORKS

Rust's rule is **aliasing XOR mutation**: at any moment a value has either any number of readers (`&T`) or exactly one writer (`&mut T`), never both. Every value has one owner; passing it *by value* moves it, and the old name can't be used again. The compiler checks this before the program runs, which is why two tasks can't write one buffer at once: the second `&mut` doesn't compile.

The same exclusion also stops a common race condition *inside one program*, check-then-act, if the check and the act are made to need the same exclusive access:

```

use std::sync::Mutex;

struct Device { open_reorder: Option<ReorderId> }

/// A reservation can only be created by `reserve` and only used once.
pub struct Reservation { device_id: DeviceId }

fn reserve(device: &Mutex<Device>, id: DeviceId) -> Option<Reservation> {
    let mut d = device.lock().unwrap();          // exclusive: no one else can check or act
    if d.open_reorder.is_some() { return None; } // check...
    d.open_reorder = Some(ReorderId::pending()); // ...and act, under the same lock
    Some(Reservation { device_id: id })
}                                                  // lock released here, after both

fn place_order(r: Reservation) { /* takes ownership */ }

// place_order(r); place_order(r); // error[E0382]: use of moved value `r`

```

- **The lock guard** is the only way to touch `Device`, so the check and the write can't be split by another task.
- **The** `Reservation` is consumed by value: one reservation buys one order, and spending it twice is a compile error, not a runtime bug.

Where it stops: ownership lives in one process's memory. Two Workers, a retried request or a second agent each have their own copy, and the compiler can't see across the network. That's why the table puts the cross-request fix in the database. The idempotency key is the same idea as the `Reservation`, enforced by a unique index instead of the compiler.

AI agents make race conditions more common: they retry, run in parallel and repeat themselves. Every action an agent can trigger needs an idempotency

key, so the second attempt is refused by the database, not by luck.

In brief: idempotency keys and the system of record

Problem. Networks fail, AI agents retry and streams deliver the same event more than once. Without protection, a retry looks like a new order, and two "ink 8%" readings become two reorders. At the same time, every system holds its own version of the facts: a token says an agent may buy, a cache says stock is available, a 15-minute SAP batch says something else. When they disagree, nothing says which one is right.

Challenge. Two questions need one answer each, every time: *has this already happened?* and *what's true right now?* An **idempotency key** answers the first.

It's a unique ID for one intended action, such as `reorder:{device_id}:`

`{low_ink_event_id}`, sent with the request and stored with its result under a unique index; a repeat with the same key gets the stored result back and nothing happens twice. A **system of record (SoR)** answers the second. It's the one system where a fact is first written, owned and kept with its history; every other system holds a copy, and when a copy disagrees, the system of record wins and the copy is corrected. The hard part is discipline: every agent, device and integration has to route its actions through the same keys and the same record, including the batch jobs that would rather write straight to their own copy.

Result. Xano is the system of record for claims, entitlements, consent, agent and device identities, reorders and agent actions; card data stays with Shopify's checkout and invoices with SAP. An agent's action counts only once it's recorded in Xano under its idempotency key. Retries return the first result instead of repeating it, a revoked permission is refused even when the agent's token still says yes, and reconciling any copy, an agent's view, a cache or a batch, is a comparison against one record rather than an argument between systems.

Idempotency, real-time streams and evaluation

Three terms carry the rest of this section.

Idempotency. An operation is idempotent when doing it twice has the same effect as doing it once. "Set stock to 12" is idempotent; "subtract 1 from stock" isn't. To make an action idempotent, the caller sends an **idempotency key**: a unique ID for the *intent*, such as `reorder:{device_id}:{low_ink_event_id}`. The server stores the key with the result. A repeat with the same key gets the stored result back and nothing happens a second time ([Stripe](#)).

Real-time data streaming. Each change is published as an event the moment it happens (a webhook, a queue message, a change-data-capture record) and consumers act on it within seconds. Streams deliver events *at least once*, sometimes out of order. That's why streaming and idempotency come as a pair: at-least-once delivery plus an idempotent consumer gives an effectively-once result.

Evaluation. Before an event causes an action, it's evaluated against current state: is the claim still valid, is consent still on, is the spending cap unspent, is the world still the way the event assumes (a conditional update on a version number)? For AI agents the same step scores the agent's decision against live data before it's allowed through. Evaluation is only as good as the freshness of what it reads.

STEP	WHAT HAPPENS	WHAT MAKES IT SAFE
1. Event	The device reports "ink 8%"; a webhook reaches the Worker with an event ID	Signed payload, event ID from the source
2. Deduplicate	The Worker derives the idempotency key and inserts it	Unique index: a repeat is refused by the database
3. Evaluate	Re-read claims, consent, cap, open reorders and stock	Read from current state, never from the event or a token
4. Act	Create the reorder	Conditional update: "only if no open reorder exists"
5. Record	Write the audit entry	The consent-aware event bus: which agent, for whom, under which claim

When the integration layer runs every 15 minutes

Many ERP integrations, SAP behind Boomi, Celigo or MuleSoft, run as **scheduled batches**: a poll every 15 minutes, IDocs collected and sent by a background job, or EDI documents (850 purchase orders, 846 inventory, 856 ship notices) exchanged in batches through a trading partner. All three platforms can run on events, but the schedule is often set by something else: SAP's batch jobs, a partner's EDI window, or API and licence limits. When it is, the ERP's view trails the stream by up to 15 minutes, and anything that evaluates against it reads stale truth.

WHAT BREAKS	EXAMPLE	WHY
Evaluation reads stale state	An agent checks stock synced 14 minutes ago and promises the last unit; SAP sold it 12 minutes ago	Step 3 above is only correct if "current state" is current
Idempotency keys don't survive the hop	A batch map drops the event ID; a failed run is retried and SAP creates every sales order twice	The real-time layer refused duplicates; the ERP never saw the key
Order collapses	A reorder and its cancellation, three minutes apart, land in one batch and are applied in file order, not event order	Batches sort by arrival, not by when things happened
Revocation lags	Consent is withdrawn or a refund issued at 10:01; the entitlement synced from SAP changes at 10:15	For 14 minutes an agent can act on a revoked right: the same snapshot problem as token extras
Agents amplify the gap	No confirmation for 15 minutes, so the agent retries, or a second agent acts on the same signal	The pending state is invisible to anyone outside the batch
Regulated data piles up	Each batch is a file of full customer records waiting on SFTP or in a staging table	A batch is a copy: it widens where PII lives and must be classified like any store

The pattern survives if the batch is demoted from authority to reconciliation:

- **Decide on the real-time path, reconcile on the batch.** Revocable and high-stakes checks read Xano claims and the Xano ledger below, fed by Shopify webhooks. The 15-minute feed corrects and reports; it never grants.
- **Carry the idempotency key end to end.** Map it into a unique external reference in SAP, so a replayed batch is refused there too, not just at the Worker.
- **Stamp freshness and set a tolerance per action.** Every synced record carries an `as_of` time. Evaluation refuses, or climbs the decision ladder to a human, when the data is older than the action allows: seconds for selling the last unit, minutes for a catalogue description.

- **Reserve, then confirm.** Hold a reservation in the real-time layer when the decision is made; the batch confirms or releases it. Show the agent "pending" so it doesn't retry.
- **Move the hot paths onto events.** Stock, price, order status and consent belong on events: IDocs set to send immediately rather than collected, SAP's event mesh, Boomi Event Streams, MuleSoft Anypoint MQ or Celigo's real-time listeners. Keep EDI batches for documents a partner batches anyway.

The rule of thumb: a 15-minute feed can tell you what happened; it can't tell an agent what's true now.

Xano as system of record: a timestamped ledger

One system has to be the answer when two disagree.

System of record (SoR). The one system where a fact is first written, owned and kept, with its history. Every other system holds a copy. When a copy and the system of record disagree, the system of record wins and the copy is corrected. A fact has exactly one system of record; a system can be the record for some facts and a copy for others.

Source of truth. The system every decision *reads* from. Here it's the same system as the record, on purpose: if agents evaluated against a copy, a permission could be granted on data the record has already changed.

ROLE	DEFINITION	WHO PLAYS IT HERE
System of record	Writes, owns and keeps the history of a fact	Xano , for claims, entitlements, consent, agent and device identities, reorders and agent actions
Source of truth	What evaluation reads before any action	Xano , the same tables, read per request
System of engagement	Where people or agents start an action	Shopify storefront and checkout, Webflow, devices, AI agents
Copy	Holds a synced or cached version for speed or reporting	Token extras, Worker caches, Shopify tags and metafields, SAP via the 15-minute batch, vector stores

Some facts have a different system of record: card data belongs to Shopify's checkout and never enters Xano; the general ledger and invoices belong to SAP. Xano holds references to those, not the facts themselves.

Why the definition matters for reconciliation. An agent's view and a permission can each drift: the agent holds a token, a cached entitlement or a fact it was told minutes ago, while the claim in Xano has changed. Reconciliation means comparing each copy to the system of record and resolving the difference one way, always in the record's favour:

AGENT'S COPY SAYS	XANO SAYS	RESULT
Allowed (token extras, cache)	Claim revoked, consent withdrawn or cap spent	Refused. The agent's copy is invalidated and the refusal recorded
Not allowed, or unknown	Claim active	Evaluate normally; refresh the agent's copy
Action done (it got no confirmation, or it retried)	Idempotency key already in the ledger	Return the stored result; nothing happens twice
Action done	No ledger entry	It didn't happen. The agent must submit it through Xano
Fact from a 15-minute batch	A newer entry by <code>occurred_at</code>	The newer entry stands; the batch difference is flagged, not applied

Shopify, SAP, Webflow, the Worker and every AI agent hold copies or send requests; none of them is the authority for claims, consent, entitlements, reorders or agent actions.

Xano keeps that authority as a **ledger**: an append-only table of events, never edited or deleted, from which current state is derived.

FIELD	FORMAT	WHY
<code>id</code>	Sequential ID assigned by Xano	Total order inside the system of record
<code>idempotency_key</code>	Unique index	A repeat from a retry, a replayed batch or a second agent is refused here
<code>occurred_at</code>	ISO 8601 in UTC, e.g. <code>2026-10-05T14:03:27.412Z</code>	When it happened at the source: used to order events
<code>recorded_at</code>	ISO 8601 in UTC, set by Xano	When the system of record learned of it: the gap to <code>occurred_at</code> is the latency, so a 15-minute batch shows up in the data
<code>source</code>	<code>shopify_webhook</code> , <code>sap_batch</code> , <code>agent:{id}</code> , <code>device:{id}</code>	Who said so
<code>actor</code> / <code>on_behalf_of</code>	Agent, device or user ID; the owner it acts for	Agents are principals, never borrowers of a person's credentials
<code>claim_id</code>	Reference to the entitlement checked	Which right allowed it, at that moment
<code>event_type</code> / <code>payload</code>	Controlled list; IDs and derived facts, not raw PII	The ledger is long-lived: minimise what goes in
<code>region</code> , <code>currency</code> , <code>language</code>	ISO 3166-1, ISO 4217, ISO 639 / BCP 47 codes	A shared register of codes, so <code>GB</code> , <code>GBP</code> and <code>en-GB</code> mean the same thing in Xano, SAP, Shopify and an agent's prompt
<code>data_class</code>	<code>none</code> , <code>pii</code> , <code>phi</code> (never <code>pci</code>)	Residency and model routing are decided per class
<code>reverses</code>	ID of the entry it corrects	Mistakes are fixed with a new entry, never by editing history

The guideline for code, AI agents and permissions pipelines:

1. **Write to the ledger first, then act.** An action that isn't in the ledger didn't happen. Agents and code call a Xano endpoint that records the intent with its key, evaluates, and only then creates the order, download or rights change.

2. **Read permissions from Xano, not from copies.** Token extras, glTF extras, Shopify tags and SAP fields are labels. The permission check reads the claim in Xano on every high-stakes request.
3. **Timestamps are ISO 8601 UTC, always.** No local times, no epoch numbers in one system and strings in another. Order by `occurred_at`; measure staleness with `now - recorded_at`; refuse when it exceeds the action's tolerance.
4. **Codes come from ISO registers.** Region, currency and language are ISO codes validated on write; free text like "UK" or "pounds" is rejected, so agents can't invent values.
5. **Copies are reconciled against the ledger, not the other way round.** A 15-minute SAP batch is recorded as entries with `source: sap_batch`; where it disagrees with what Xano already recorded, the difference is flagged for review, not overwritten.
6. **Revocation is an entry.** A withdrawn consent or a refund is appended the moment it's known and takes effect on the next evaluation, with no wait for a token to expire or a batch to run.
7. **Audit is a query.** "Which agent, acting for whom, did what, under which claim, with which data class" is a read of the ledger, not a forensic exercise across logs.

Permissions: claims and entitlements, not token extras

Xano's authentication tokens are encrypted JWE tokens, and their *extras* can carry data such as a user's role (Xano). Extras are written when the token is created and last until it expires, so they're a snapshot.

USE	WHERE IT LIVES	WHY
Fast, low-risk checks: role for the interface, tenant, plan tier	Token extras	No lookup per request; encrypted, so the client can't read or change them
Anything revocable or high-stakes: orders, payments, firmware downloads, rights changes	Claims and entitlements in Xano tables, re-checked on every request	A refund or a withdrawn consent can't be pulled back out of an issued token
Device and agent actions	The device or agent record, the owner's consent and the spending cap, checked per request	A device or agent token never carries purchase rights by itself
glTF <code>extras</code> (game-object IDs, SKUs)	The model file	Written by whoever edits the file: fine as a label, never a permission

PII, PCI and PHI: where each class may live

Classify data before designing where it flows. The class decides which systems may hold it, not convenience.

CLASS	WHAT IT IS	EXAMPLES HERE	MAY LIVE IN	MUST NEVER BE IN	KEY CONTROLS
PII (personal data)	Anything that identifies a person: GDPR, CCPA, and state health-data laws	Name, email, address, device owner, a home's temperature history	Xano, in encrypted fields; Shopify customer records	Logs, URLs, token extras beyond an ID, themes, AI prompts beyond what the task needs	Minimise, consent, encrypt at rest, access by claim, deletion on request
PCI (cardholder data)	Card number, security code, track data, under PCI DSS v4.0.1	Card details at checkout	Only the payment provider's checkout (Shopify's), which hands back tokens	The Worker, Xano, logs, AI, devices: anywhere you control	Stay out of scope: never touch card data. The simplest assessment (SAQ A) now requires confirming your site isn't open to malicious scripts (PCI SSC), which the edge head function's CSP supports
PHI (health information under HIPAA)	Health information held by a covered entity or its business associate	A clinic's patient readings	Only vendors with a signed business associate agreement: Xano offers one on Enterprise plans (Xano); Cloudflare signs one for Enterprise customers, for listed services (Cloudflare)	Shopify: it signs no agreement, and its acceptable-use policy prohibits PHI (HIPAA Journal). Any AI model without an agreement	Agreement first, then encryption, audit logs, minimum necessary access

A consumer thermometer sold directly to households is usually **not** HIPAA PHI, because the seller isn't a covered entity. Its readings are still **consumer health data**: Washington's My Health My Data Act has required consent for collecting

and sharing it since 31 March 2024 (Goodwin), and the FTC's Health Breach Notification Rule covers health apps and devices outside HIPAA (FTC). Design it as PII with health-grade consent.

AI-shaped infrastructure

Infrastructure as a service was designed for people and programs. When AI agents act inside it, two things change: the agent becomes a **principal** with its own identity, and data has to be **shaped** before it reaches a model.

CONCERN	TRADITIONAL INFRASTRUCTURE	AI-SHAPED INFRASTRUCTURE
Identity	People sign in; services hold API keys	Each agent has its own identity and claims, like a device: never a person's credentials, revocable on its own
Permissions	Role-based, checked at sign-in	Checked per tool call against Xano entitlements and caps; high-stakes actions climb the decision ladder to a human
Data to the model	Not applicable	Shaped at the Worker: IDs and derived facts ("ink low: yes"), never raw PII; card data never; PHI only to a model provider with a business associate agreement and no data retention
Data stores	Databases and files	Also prompts, outputs, logs, caches and vector stores. Classify them all: personal data in embeddings is hard to find and delete, so store pseudonymous IDs there
Retries and parallelism	Occasional	Constant: idempotency keys on every action
Audit	Who logged in	Which agent, acting for whom, called which tool, with which data class, under which claim. This is the consent-aware event bus of the Higher-Order Stack
Residency	Per application	Per data class and region, routed at the Worker before any model call

The rule of thumb: the model is the least trusted reader in the system. Give it the smallest shape of data that lets it do the job, and let the server, not the model, hold the keys.

Recent cases, 2025–2026

The AI privacy cases in healthcare so far aren't about models being hacked. They're about consent, vendors and staff behaviour. The lawsuits below are allegations, not findings.

CASE	WHAT HAPPENED	LESSON	RULE
Sharp HealthCare, lawsuit filed 26 November 2025 in San Diego	Alleged: an ambient AI scribe (Abridge) recorded patient visits without consent, sent recordings to outside servers, and wrote chart notes falsely stating patients had consented; plaintiffs estimate over 100,000 encounters (Becker's)	Consent is an event captured at the moment of recording. Software may record it, never assert it	SB-21
Sutter Health and Memorial Healthcare Services, lawsuit filed 14 April 2026 in federal court	Alleged: the same tool recorded conversations without proper consent and sent the audio out for processing, under California's wiretap and medical-confidentiality laws and the federal Wiretap Act. HIPAA isn't claimed: the vendor works under business associate agreements (HIPAA Journal)	A business associate agreement is necessary but not enough: state consent and wiretap laws apply too	SB-21
HClactive, breach July 2025	A hacker copied files from an "AI-powered" health-plan administrator; the count reported to HHS grew from 501 to 3,056,950 people by January 2026, with Social Security numbers and medical records exposed (The HIPAA E-Tool). The cause was ordinary hacking, not the AI	One vendor serving many clients concentrates millions of records: send vendors IDs and derived facts, not full records	SB-22
Shadow AI, Netskope 2025 healthcare report	71% of healthcare workers still use personal AI accounts for work, and 81% of data-policy violations involved regulated data such as PHI (Medical Economics)	Route AI through approved providers at the Worker, and test with canary values	SB-22

HHS's Office for Civil Rights, which enforces HIPAA, has made the vendor rule plain: covered entities can't hand their HIPAA duties to an AI vendor, the business associate agreement comes before the service starts, and if the vendor is breached the provider still notifies patients. A missing risk analysis is still the most-cited failure ([HIPAA Journal](#)).

The same lesson reaches the household thermometer, outside HIPAA: capture consent before the data is collected, and keep the proof on the server.

11. Rights metadata as lightweight DRM

XMP rights fields don't lock an image; they declare who owns it and on what terms, in a form machines act on. Real DRM encrypts content. Rights metadata is closer to a label that travels with the file: it can be stripped, but a crawler, DAM or training pipeline that respects it can read the terms without a human.

What each machine does with it:

- **Google Images** shows a *Licensable* badge when an image has a Web Statement of Rights (`xmpRights:WebStatement`). It also reads Creator, Credit Line, Copyright Notice, Licensor URL and Digital Source Type. Where XMP and the page's structured data disagree, Google uses the structured data (Google Search Central).
- **AI and data-mining pipelines** can read `plus:DataMining` . Its controlled values include *Prohibited*, *Allowed*, and *Prohibited except for search engine indexing*, which the IPTC says rules out other uses such as AI/ML training (IPTC 2025.1).
- **DAMs** index the full packet, so rights and usage terms show up in search and on download.

For agencies, the minimum useful set is: Copyright Notice, Creator, Credit Line, Web Statement of Rights pointing to a real licence page, Licensor URL, and a Data Mining value. Put the same licence URL in the page's JSON-LD (`license` , `acquireLicensePage`) so both readers agree.

Content Credentials go a step further. Photoshop's export dialog now has a *Content Credentials (Beta)* panel. These are cryptographically signed provenance records (the C2PA standard), so a reader can tell if they were tampered with, which plain XMP can't show. They complement XMP rather than replacing it.

12. Favicons

A favicon is the most widely copied image a site has. It appears in tabs, bookmarks, search results and home screens, so it carries the brand further than any page image. Google requires it to be square and at least 8×8 px, recommends larger than 48×48, and accepts ICO, PNG, JPEG, GIF, BMP and TIFF ([Google Search Central](#)).

SIZE	USED FOR	BEST FORMAT
16, 32, 48 px	Browser tabs, <code>/favicon.ico</code> fallback	ICO holding all three, plus a 32 px PNG
180 px	iPhone and iPad home screen (<code>apple-touch-icon</code>)	PNG
192 px	Android home screen	PNG
512 px	App install and splash screens	PNG, from a 512 px or larger source

PNG over JPG. JPG has no transparency, so a JPG icon always sits on a solid square. Webflow handles this for you: the icons it generated for game11ty were real RGBA PNGs, even though their file names end in `.jpg`. Declare each one with an accurate `type` and `sizes`, and serve the real extension so the server sends the right content type.

Upscaling costs. Webflow made the 512 px icon by enlarging a 256 px source. The result is 355 KB, against 5 KB for the 32 px icon, and it looks soft at full size. Start from the largest size you need.

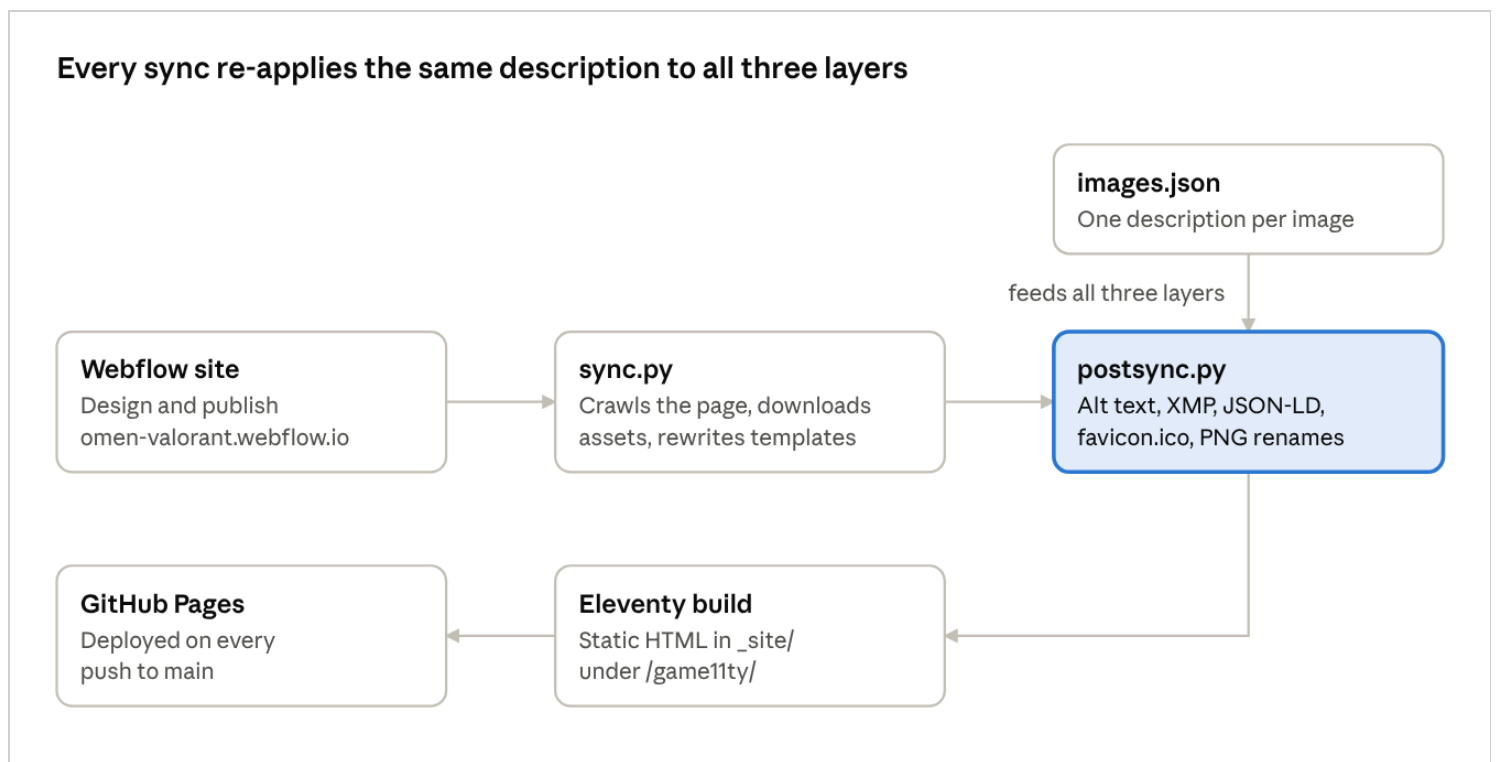
Brand permission. game11ty first shipped HyperX's logo as its favicon, copied over by the Webflow sync. Because a favicon is shown so widely, a third-party logo there implies an endorsement. It was replaced with an original crop with no logo.

Metadata on icons. Rights XMP on a 32 px icon is 47% of the file. That's still only about 2.5 KB, and icons are the images most often lifted, so keep a

minimal set: alt text, copyright and the rights URL.

13. Case study: game11ty

game11ty is a one-page OMEN x Valorant site designed in Webflow, converted to static Eleventy pages by a Python script, and published on GitHub Pages. It was built on 3 October 2026, and the alt text, XMP and favicon work all happened that day.



`sync.py` regenerates the templates from Webflow on every run, so anything added on top would be lost. `postsync.py` re-applies it from one file, `images.json`, which holds a single description per image. That one source is what keeps the three layers in agreement.

CHECK	BEFORE	AFTER
Images with XMP	6 of 12, mostly tool names and IDs	All 9 page images, 5 favicons and the PDF
Images with an XMP alt text	1, describing a laptop when the photo shows a desktop	All 9, matching the page
Rights fields (copyright, WebStatement, LicensorURL)	None	Every file, linking to a rights page
HTML <code>alt</code> accuracy	2 wrong: the OMEN logo labelled "Valorant Champions", one photo given another's caption	All 9 corrected
JSON-LD <code>ImageObject</code>	None	10 entries with description, licence and copyright
Favicon	HyperX logo, JPG declared as <code>image/x-icon</code>	Original icon, 5 PNG sizes plus <code>favicon.ico</code>

The rights values are deliberately generic ("© Rights Holder... demonstration rights metadata for agencies"). The site demonstrates the fields; it doesn't claim to own the brand imagery.

14. One image, three channels

The same pixels go out as a social ad, an email hero and a page image. Each channel reads a different layer, so one description has to be placed three ways. Using the game11ty share image as the example:

CHANNEL	WHAT THE MACHINE READS	WHAT SURVIVES	WHERE THE DESCRIPTION GOES
Social ads and link previews	<code>og:image</code> , <code>og:image:alt</code> , <code>twitter:image:alt</code> from the page	Platforms typically re-encode uploads, so embedded XMP is usually lost	<code>og:image:alt</code> on the page, plus the alt-text field in the ad tool
Email	The <code></code> in the email HTML	Many clients block images until the reader allows them, and show the alt text instead; clients don't read XMP	A complete <code>alt</code> that works as copy on its own
Page	<code>alt</code> , the section's heading, JSON-LD, and the file's XMP	All three layers	<code>images.json</code> → <code>postsync.py</code>

A 1×1 tracking pixel in an ad or email is the opposite case: it carries no meaning, so give it `alt=""`. Screen readers then skip it, and parsers don't mistake it for content.

Section IDs paired with H1s as an index

An `alt` is read in the context of the section around it. The section's `id` is the machine address (a deep link such as `/game11ty/#complete-your-setup`), and its heading is the human label. Together they index the page, and the alt text should add to that label, not repeat it.

SECTION ID	H1 + H2	IMAGE	ALT TEXT
#omenhero	Omen x Valorant / An Official Partner of the Valorant Champions Tour	Valorant logo	Valorant Champions Paris logo
#omen-35L-valorant	No H1 / OMEN 35L Valorant Gaming Desktop Special Edition	Desktop	OMEN 35L Valorant Gaming Desktop Special Edition with two lit front fans
#follow-the-action	No H1 / Follow the Action. Fuel Your Game.	Emblem	Red Valorant Champions emblem
#complete-your-setup	Complete / Your Valorant Setup	3 product photos	Bundles for FPS Games: HyperX headset, microphone, keyboard and mouse on a desk (and two more)
#why-omen-valorant	Why / Omen x Valorant	Partnership art	OMEN, HyperX and Riot Games partnership artwork for Valorant

This shows the real gap in game11ty: not how many H1s it has, but what they say. Two of its three H1s ("Complete", "Why") only make sense when read with the H2 below them, and two sections have no H1 at all, so a parser building an outline gets "Complete" and "Why" as top-level topics.

Give every section an `id` and a heading that reads on its own. A section may open with its own H1 when it stands alone as a topic, and dynamic pages can load several H1s as views or sections arrive. HTML allows this as long as the H1s aren't nested inside one another, though a single H1 per page is still the common recommendation ([MDN](#)). Whatever loads, keep the heading order logical (H1, then H2, with no skipped levels) so screen readers and parsers can still build a clean outline:

```
<section id="complete-your-setup">

  <h2>Complete your Valorant setup</h2>

</section>
```

The ad or email then links to `/game11ty/#complete-your-setup`. Someone clicking through lands on the section whose heading, alt text and file metadata all say the same thing.

15. 3D mesh data: CLO wraps and game objects

A 3D garment is a stack of images on a mesh. The fabric "wrap" is a set of UV-mapped textures (base colour, normal, sampler maps) laid over mesh geometry. That makes it two kinds of data, and only the textures have the XMP habits described above. glTF, the standard format for web and game 3D, has its own metadata slots, and they're usually left empty or filled in wrongly.

The CLO jean jacket and avatar on the board, as exported through Adobe Substance 3D Stager:

FILE	SIZE	MESH DATA	TEXTURES	WHAT ITS METADATA SAYS
jeanjacket.glb	8.2 MB	152 meshes, 98,560 vertices, 6.1 MB	19 embedded, 1.7 MB; 7 carry XMP	asset.copyright : "2025 (c) Adobe Inc."
maramodel.glb (avatar)	2.1 MB	12 meshes, 24,303 vertices	7 embedded	asset.copyright : "2025 (c) Adobe Inc."; skeleton joints as plain nodes, no skin
jeanjacket.gltf + .bin + 21 loose images	6.7 MB	same jacket, split into separate files	8 of 21 carry XMP	same Adobe copyright

Three things stand out:

- **The exporter claimed the copyright.** Stager wrote Adobe's own copyright line into `asset.copyright` on all three models. A crawler or marketplace reading that field would credit Adobe, not the designer. Overwrite it on export.
- **Names carry the lineage, not the metadata.** Node and material names record where each part came from: `JEANALL2%2Eroblox_n3d` (a Roblox-sourced scene), `Denim_Lightweight_FRONT_4584` (a CLO fabric), and `Mara:` and `Feifei_hair` (CLO avatar parts). That's useful tracking data, but it's informal and gets lost on rename.
- **No game-object IDs.** None of the 189 nodes has `extras` or an extension. A game engine or tracking system has nothing stable to hang analytics, SKUs or ownership on.

Where 3D metadata goes in glTF

SLOT	SCOPE	USE IT FOR
<code>asset.copyright</code> , <code>asset.generator</code>	Whole file	Rights holder and the tool that made it
<code>KHR_xmp_json_ld</code>	Whole file, or one scene, node, mesh, material or image	Full XMP (<code>dc:</code> , <code>xmpRights:</code> , IPTC) as JSON-LD; object-level packets override the file-level one
<code>extras</code>	Any object	Your own tracking fields: game-object ID, SKU, garment piece
Texture XMP	Each embedded JPG or PNG	Alt text and rights for the wrap itself

`KHR_xmp_json_ld` is a ratified Khronos extension ([spec](#)). It lets the same rights fields used on game11ty's images travel inside the 3D file, down to a single garment piece. Here is the jacket with a file-level packet and one tracked game object:

```

{
  "asset": { "version": "2.0", "copyright": "© Rights Holder", "generator": "CLO → Substance 3D",
  "extensionsUsed": ["KHR_xmp_json_ld"],
  "extensions": { "KHR_xmp_json_ld": { "packets": [{
    "@context": { "dc": "http://purl.org/dc/elements/1.1/",
      "xmpRights": "http://ns.adobe.com/xap/1.0/rights/" },
    "@id": "",
    "dc:title": "Lightweight denim jacket",
    "dc:description": "Light-wash denim jacket on a CLO avatar, 152 garment pieces",
    "xmpRights:WebStatement": "https://example.com/rights/"
  ]}},
  "nodes": [{
    "name": "Cloth_mesh_n3d",
    "mesh": 0,
    "extras": { "gameObjectId": "jacket-front-left", "sku": "DJ-4584", "fabric": "Denim_Lightwe",
    "extensions": { "KHR_xmp_json_ld": { "packet": 0 } }
  }]
}

```

The IDs and SKU are placeholders. Reading it back takes a few lines of Python, because a `.glb` keeps its JSON in the first chunk after a 20-byte header:

```
import json, struct

data = open("jeanjacket.glb", "rb").read()

length = struct.unpack("<I", data[12:16])[0]          # JSON chunk length

gltf = json.loads(data[20:20 + length])

print(gltf["asset"].get("copyright"))                  # who the file says owns it

for node in gltf["nodes"]:

    if "extras" in node:                               # tracked game objects

        print(node["name"], node["extras"].get("gameObjectId"))
```

Run against the real jacket today, the first line prints "2025 (c) Adobe Inc." and the loop prints nothing. That's the gap a 3D pipeline needs to close: the rights field is wrong and the game objects are untracked.

Tool references: from Illustrator art to a 3D model

Artwork often starts as a vector in Illustrator, gets wrapped onto a model in Substance or Cinema 4D, and ends up in an After Effects render or a web viewer. Each hand-off is a chance to keep or lose the description and rights.

TOOL	WHAT IT MAKES	METADATA IT KEEPS OR WRITES	WHERE IT'S LOST
Illustrator (3D and Materials panel)	Extruded and revolved 3D objects with Substance materials; exports glTF, USDA, USDZ and OBJ since version 27.0 (Adobe)	The <code>.ai</code> file's own XMP	OBJ export keeps only the object's colour, not its Substance materials
Substance 3D (Painter, Stager)	Texture sets (base colour, normal, roughness) and staged scenes exported as glTF/GLB (Stager formats)	XMP in some exported textures; <code>asset.generator</code>	Stager wrote "2025 (c) Adobe Inc." into <code>asset.copyright</code> on the jacket and avatar
Cinema 4D	Modelling, UV unwrapping, animation; glTF 2.0 export with an optional Draco setting in recent versions (guide)	Object and material names	Anything not mapped to a glTF field
Cinema 4D Lite (ships with After Effects)	A limited Cinema 4D for building 3D scenes that render inside After Effects through the Cineware plug-in (Adobe , Maxon)	Lives as a <code>.c4d</code> layer in the AE project	The 3D data stays in the AE project; renders carry only AE's XMP
After Effects	Video and image-sequence renders	With <i>Include Source XMP Metadata</i> on, writes markers, comments, project XMP and every source file's XMP into the render; off, only a unique ID (Adobe)	Off by default in many output modules: check it

The weak points are the exporters, not the tools. Set the copyright and description at the last export, check `asset.copyright` in every GLB, and turn on *Include Source XMP Metadata* in After Effects when a render has to carry rights.

Video poster images

A video's poster frame is an ordinary JPG or PNG, so it carries XMP rights like any other image. It's also the part of a video that machines see first. A `<video>` element has no `alt`, crawlers index the thumbnail rather than decoding the video, and social previews show the poster. That makes the poster the video's rights label on the web.

```
<video controls poster="/media/omen-teaser-poster.jpg"
      aria-label="OMEN x Valorant teaser: the OMEN 35L desktop on stage in Paris">
  <source src="/media/omen-teaser.mp4" type="video/mp4">
</video>

<script type="application/ld+json">
{ "@context": "https://schema.org", "@type": "VideoObject",
  "name": "OMEN x Valorant teaser",
  "description": "The OMEN 35L desktop on stage in Paris",
  "thumbnailUrl": "https://example.com/media/omen-teaser-poster.jpg",
  "uploadDate": "2026-10-03",
  "license": "https://example.com/rights/" }
</script>
```

The file names and dates are placeholders. Write the same description and rights into three places: the poster's XMP, the `VideoObject` JSON-LD (`thumbnailUrl` points at the poster), and the video file itself. For that last one, turn on *Include Source XMP Metadata* when rendering from After Effects. If the video is re-encoded or stripped, the poster still carries the rights.

Video optimization in Adobe Media Encoder

Media Encoder is where a video's size and its metadata are decided together. The *Metadata* button in Export Settings offers two ways to keep XMP: *Embed in Output File* or *Create Sidecar File*, a separate `.xmp` next to the video (Adobe).

Users on Adobe's forum report that embedding is greyed out for H.264 MP4 and available for QuickTime ([Adobe community](#)). So the most common web format often ships with no rights inside it.

SETTING	WEB RECOMMENDATION	WHY
Format	H.264 (MP4) for reach; H.265 or AV1 where supported	Smaller files at the same quality
Bitrate encoding	VBR, 2 pass	About 10% better quality for the same size, at roughly twice the encode time (guide)
Maximum bitrate	About 2× the target	Leaves room for fast motion without raising the average
Target bitrate	About 16 Mbps for streaming uploads; lower for embedded web video	Platforms re-encode anyway; 32–40 Mbps is master quality (guide)
Metadata	<i>Create Sidecar File</i> for MP4; <i>Embed</i> for QuickTime masters	Keeps the rights whether or not the container can hold XMP
Poster	Export a still frame as JPG and give it rights XMP	The frame crawlers and social previews index

For game footage like the Valorant streams on the board, fast motion is what eats bitrate. Keep 2-pass VBR, and put the rights in the sidecar, the poster and the `VideoObject` JSON-LD. Don't rely on the MP4 alone.

Draco compression

[Draco](#) is Google's open-source codec for compressing mesh geometry: vertex positions, normals, UVs and triangles. It isn't a media type of its own. In glTF it's carried by the `KHR_draco_mesh_compression` extension ([Khronos](#)), and the file is still served as `model/gltf-binary` (`.glb`) or `model/gltf+json` (`.gltf`).

I ran the jean jacket through Draco with `gltf-transform` to see what it costs and what it keeps:

	ORIGINAL	DRACO
File size	8.19 MB	2.86 MB (65% smaller)
<code>asset.copyright</code>	2025 (c) Adobe Inc.	Kept
<code>asset.generator</code>	Adobe Substance 3D Stager	Replaced by glTF-Transform v4.5.1
Node names (189)	<code>JEANALL2%2Eroblox_n3d</code> ...	Kept
Textures with XMP	7 of 19	7 of 19 (textures aren't touched)
Viewer needs	Any glTF viewer	A Draco decoder (<code>extensionsRequired</code>)

Draco only compresses geometry, so the mesh data shrinks and the textures and their XMP pass through untouched. It does rewrite `asset.generator`, and the one tool-specific extension in the jacket (`EXT_materials_specular_edge_color`) triggered a warning. Run your rights and tracking fields through the compressor and check them afterwards, as you would for a WebP conversion.

Two layers, two owners. A 3D file splits the same way an image does. The geometry underlayer is open and Google-optimised: Draco compresses it and any glTF viewer with a decoder can read it. The shading layer is Adobe-bound. Substance materials are procedural inside Adobe's tools, and leave them only as baked textures plus standard PBR settings. In the jacket, the Khronos `KHR_materials_specular` setting survived Draco on 12 of 13 materials. Stager's own `EXT_materials_specular_edge_color` declaration was dropped, though no material used it. Rights for the geometry belong in glTF fields. Rights for the look belong in the texture XMP and the material's `KHR_xmp_json_ld` packet, because that's the part that stays tied to Adobe's material model.

16. Checklist for agencies and media teams

- ☐ Keep one description per image in a single source file, and generate the `alt`, XMP and JSON-LD from it
- ☐ Write alt text that adds to the section's heading instead of repeating it
- ☐ Give every section an `id` and a heading that reads on its own; separately loaded sections may each bring an H1 (never nested), with no skipped heading levels
- ☐ Embed Copyright Notice, Creator, Credit Line, Web Statement of Rights, Licensor URL and a Data Mining value
- ☐ Point the Web Statement of Rights and the JSON-LD `license` at the same live rights page
- ☐ Set `og:image:alt` for every share image, and fill the alt-text field in ad tools
- ☐ Write email alt text that works as copy when images are blocked; give tracking pixels `alt=""`
- ☐ Turn on metadata keeping in every encoder and optimiser (e.g. `cwebp -metadata xmp`)
- ☐ Strip edit history and padding from web copies; keep the full packet in the DAM
- ☐ Serve favicons as PNG and ICO from a 512 px source, with no third-party logos
- ☐ Re-check the live site with a parser after every build or sync

exiftool commands

```
# Read the description and rights fields

exiftool -AltTextAccessibility -Description -Rights -WebStatement -LicensorURL -DataMining photo.jpg

# Write a minimal rights set (JPG, PNG and WebP)

exiftool -overwrite_original \

    "-XMP-iptcCore:AltTextAccessibility=Player at an OMEN desktop with red-lit fans" \

    "-MWG:Description=Player at an OMEN desktop with red-lit fans" \

    "-MWG:Copyright=© Rights Holder" "-MWG:Creator=Rights Holder" \

    "-XMP-xmpRights:WebStatement=https://example.com/rights/" \

    "-XMP-plus:LicensorURL=https://example.com/rights/" photo.jpg

# Web copy: drop edit history, keep everything else

exiftool -overwrite_original -XMP-xmpMM:all= photo.jpg
```

Sources

- [IPTC Photo Metadata Standard 2025.1](#)
- [Google Search Central: image license metadata](#)
- [Google Search Central: favicons in search results](#)
- [WebP container specification](#)
- [Khronos KHR_xmp_json_ld](#)
- [Khronos KHR_draco_mesh_compression](#) and [Google Draco](#)
- [Adobe: XMP metadata in After Effects](#)
- [Adobe: Cinema 4D and Cineware in After Effects](#) and [Maxon: Cineware](#)
- [Adobe: export 3D vector artwork in Illustrator](#)

- [Adobe: Substance 3D Stager import and export formats](#)
- [Adobe: Media Encoder export settings reference](#), [Adobe community: XMP embed disabled for MP4](#) and [Annenberg Digital Lounge: compress video for the web with Media Encoder](#)
- [CRM Sync knowledge base: Asset Management, Security and AI \(companion article\)](#)
- [Wikipedia: OpenAI–HuggingFace incident](#)
- [Cloudflare: allow traffic from IPs in an allowlist](#), [Xano: encrypting fields](#) and [Xano: Addons](#)
- [Wikipedia: List of file signatures](#), [Gary Kessler: File Signatures Table](#), [MDN: MIME types](#) and [WHATWG MIME Sniffing](#)
- [Shopify: stagedUploadsCreate](#), [Shopify: fileCreate](#) and [Webflow: Upload Asset](#), [WordPress REST API: Media](#)
- [Shopify: app proxies](#), [WordPress: HTTP API](#), [Astro: environment variables](#) and [Next.js: environment variables](#)
- [RFC 9309: Robots Exclusion Protocol](#), [RFC 9116: security.txt](#), [Ilmstxt.org](#) and [Google: qualify outbound links](#)
- [Helmet](#), [Cloudflare Workers: set security headers](#) and [Cloudflare community: Shopify and the Cloudflare proxy](#)
- [Hacktron: Hacking OpenAI](#)
- [Cloudflare: mutual TLS](#), [European Commission: CRA reporting](#), [GOV.UK: consumer connectable product security](#), [Shopify: Functions languages](#) and [Shopify Help: product media types](#)
- [PCI SSC: SAQ A eligibility](#), [Xano for Enterprise](#), [Cloudflare HIPAA FAQs](#), [HIPAA Journal: Shopify](#), [Goodwin: My Health My Data Act](#) and [FTC: Health Breach Notification Rule](#)
- [Becker's: Sharp HealthCare lawsuit](#), [HIPAA Journal: Sutter and Memorial lawsuit](#), [The HIPAA E-Tool: HClactive](#), [Medical Economics: Netskope](#)

healthcare report and HIPAA Journal: March 2026 breach report

- Stripe: idempotent requests
- How to export Cinema 4D to GLB
- Measurements and parser output: game11ty files, the live site and the CLO jacket and avatar GLBs, 3 October 2026